

Claude e Claude Code in pratica

Dal prompting agli agenti

Simone Scannapieco

Corso base per Tecnodata Trentina S.r.l., Trento, Italia

Modulo 4 — Un agente nel repository

Quando qualcosa va storto

Quattro modi di andare storto

Le quattro situazioni della pagina delle best practice, e che cosa fare in ciascuna

Fin qui il modulo ha mostrato come lavora Claude Code; qui, che cosa fare quando non va. Se una sessione non va come previsto, dice la pagina delle best practice di Claude Code, è **quasi sempre** una di queste quattro situazioni

1. Ha costruito **altro** da ciò che avevate in mente: gli manca un'informazione che leggendo il repository non può avere
   → ditegli **ciò che non vede**: il file, il sintomo, com'è il «risolto»
2. Ha detto di aver **finito**, e non era del tutto così: di solito non aveva modo di controllare il proprio lavoro
   → dategli un **controllo** da eseguire, che risponde «passa» o «fallisce»
3. Ha fatto ciò che nel repository **non deve mai** succedere, o ha saltato un passo che deve succedere **sempre**: un prompt, da solo, non può garantirlo
   → **imponetelo**: regole di permesso e hook li fa rispettare Claude Code, qualunque cosa decida Claude
4. Ha fatto bene, ma con più **tempo**, **token** o richieste di permesso di quanto valesse il compito
   → `/clear` fra un compito e l'altro, perché ogni richiesta rimanda tutta la conversazione; e una **modalità** di permesso decisa una volta per il lavoro

A voce: quale di questi errori vi è già capitato con Claude Code?

Fermare, tornare indietro, ripartire

Quando la sessione devia: i consigli di Anthropic, e quando usarli

- **Esc**, appena Claude devia: lo ferma a metà azione e conserva tutto fino a quel punto, così correggete la rotta **senza perdere lavoro**
- Dopo **due correzioni fallite**, non tentate la terza: `/clear` e un prompt migliore, che metta dentro ciò che avete imparato — funziona meglio **quasi ogni volta**
- Appena qualcosa va per il verso sbagliato: di nuovo in **plan mode**, e si ripianifica, invece di correggere la rotta a metà strada
- `/rewind`, o **Esc due volte** a prompt vuoto: si torna a un punto precedente, con il **codice**, la **conversazione** o tutti e due
- Un file cambiato **solo da comandi Bash** non torna: per quello, **git**. E ciò che ha toccato **sistemi remoti** — un database, un'API, un deploy — non torna indietro
- Una **configurazione rotta**: `claude doctor` stampa una diagnostica in sola lettura, senza aprire una sessione; `claude --safe-mode` parte con tutte le personalizzazioni spente — `CLAUDE.md`, skill, plugin, hook, server MCP — per diagnosticarla

Il contesto lungo

Nelle sessioni lunghe il contesto si riempie, e si compatta

- Lavorando, il contesto si riempie e Claude lo **compatta** da solo: le istruzioni date all'inizio della conversazione **possono andare perse**
- Sopravvivono il `CLAUDE.md` alla radice del progetto e la **memoria automatica** (auto memory); le istruzioni date solo in conversazione, forse no
- Il *context rot*: più token ci sono nella finestra di contesto, meno il modello ne richiama le informazioni **con precisione**; e una compattazione troppo aggressiva può perdere un dettaglio che conta più tardi
- `/compact` con un'indicazione: il riassunto tiene **ciò che scegliete voi**. Un file così grande da riempire subito il contesto ferma la compattazione automatica: chiedete di leggerlo **a pezzi**, un intervallo di righe o una funzione

Il rischio di errori che si sommano

Un agente lavora per molti turni: un errore può portarsi dietro i passi che seguono

- Gli agenti usano strumenti per molti turni e cambiano lo stato dell'ambiente: per Anthropic gli errori possono **propagarsi e sommarsi**
- Uno strumento sbagliato, quello giusto con i **parametri** sbagliati, troppo pochi strumenti, o una risposta **letta male**
- Il conto di Anthropic, sulle valutazioni degli agenti: con il **75 %** di successi per tentativo, tre tentativi riescono tutti circa nel **42 %** dei casi
- Lo stesso conto per i passi di un compito, come nella parte «Che cos'è davvero un agente»: se ogni passo riesce nel 95 % dei casi, venti passi di fila riescono tutti soltanto in circa il **36 %** dei casi — l'aritmetica è nostra: $0{,}95^{20} \approx 36\,\%$

Per questo: fermarlo presto, e dargli un controllo da eseguire.

Partite dal diff

Controllare che cosa ha fatto, non che cosa dice di aver fatto

- Partite dal **diff**, non dal riassunto di Claude su ciò che ha fatto
- Chiedete le **prove**: il comando eseguito e il suo output; «tutti i test passano», da solo, dice molto poco
- Un test si può **allentare** di nascosto, perché passi comunque: nel diff, guardate anche i test
- **Meno avete guardato, più verificate**
- In uno studio di Anthropic il **77 %** degli analisti ha usato gli argomenti dell'agente, a volte parola per parola; solo il **15 %** lo ha indicato come fonte. Ciò che usate dell'agente diventa vostro: **controllatelo prima**

Produttività reale o marketing?

Sette studi di terze parti, in ordine di tempo, ciascuno con la sua fonte e la sua data

studio, fonte e data	chi, e come	che cosa ha trovato
Peng et al. (arXiv, 2023)	esperimento controllato, con e senza GitHub Copilot	il compito finito il **55,8 %** più in fretta
Paradis et al. (arXiv, 2024)	trial randomizzato, 96 ingegneri di Google	circa il **21 %** di tempo in meno, con un intervallo ampio
Cui et al. (Microsoft Research, giugno 2025)	tre esperimenti, 4.867 sviluppatori	**+26,08 %** di compiti completati
METR (arXiv, luglio 2025)	trial randomizzato, sviluppatori open source esperti	misurato: il **19 %** di tempo in più; previsto il 24 % in meno, stimato a fine studio il 20 % in meno
MIT NANDA (luglio 2025; il PDF è su mlq.ai)	rapporto preliminare: oltre 300 iniziative, interviste in 52 organizzazioni	il **95 %** delle organizzazioni senza alcun ritorno dalla GenAI
DORA (Google Cloud, settembre 2025)	un sondaggio	il 90 % usa l'AI, oltre l'80 % si sente più produttivo; il **30 %** si fida poco o nulla del codice generato
METR (blog, febbraio 2026)	esperimento nuovo, due gruppi di sviluppatori	dal 4 al 18 % di tempo in meno; per METR stesso un **segnale inaffidabile**

Misuratelo voi

Gli studi non si mettono d’accordo: la misura che conta è sui vostri compiti

- Gli studi vanno dal **55,8 %** più in fretta al **19 %** di tempo in più; e nello studio del 19 % in più gli sviluppatori, a fine studio, stimavano il 20 % di tempo **in meno**. Il più recente, per METR stesso, è un **segnale inaffidabile**, una prova molto debole
- **Il tempo:** dall’inizio al lavoro **verificato**, non alla prima risposta
- **Le rilavorazioni:** quante volte siete tornati indietro — Esc, `/rewind`, `/clear` — e che cosa avete rifatto a mano
- **Che cosa avete controllato:** il diff, i test eseguiti, il comando e il suo output
- Un prima e un dopo, sullo **stesso tipo di compito**: misuratelo voi

Fermatelo presto, tornate indietro, ripartite pulito, e misurate se vi ha fatto risparmiare tempo davvero.